

Enci/ VENTAS

Guía de Solución de Problemas

Diagnóstico, contención, escalamiento y cierre de incidentes frecuentes

Versión 1.0

Proyecto Enci / Enci Ventas

Aplicación de Automatización de Fuerza de Ventas

Julio 2026

DIAGNOSTICO Causa probable	SOPORTE Accion correctiva	ESCALAMIENTO Incidente critico	PLM UTEM Entrega tecnica
-------------------------------	------------------------------	-----------------------------------	-----------------------------

Responsable de la documentación

Documento elaborado por el Grupo PLM de la Universidad Tecnológica Metropolitana (UTEM). Líder: Joshua Flores. Integrantes: Falon Berrios, Caroline Palacios y Eliseo Poblete.

Naturaleza del documento

Esta guía establece criterios de revisión para problemas frecuentes del MVP web avanzado Enci/ Ventas, su API, autenticación, permisos, despliegue, importación CSV, IA RAG parcial y aplicación Flutter nativa parcialmente realizada. Su finalidad es permitir una atención ordenada, reproducible y trazable, sin transformar limitaciones documentadas o componentes parciales en obligaciones de desarrollo adicional.

Tabla de Contenidos

1. Introducción
2. Principios de diagnóstico
3. Matriz general de problemas frecuentes
4. Códigos de error y señales del sistema
5. Diagnóstico de autenticación y acceso
6. Diagnóstico de permisos y roles
7. Diagnóstico del frontend web
8. Diagnóstico del backend API
9. Diagnóstico de datos e importación CSV
10. Diagnóstico de IA RAG parcial
11. Diagnóstico de app Flutter parcial
12. Validaciones rápidas y comandos útiles
13. Criterios de escalamiento
14. Cierre y registro del problema

1. Introducción

Enci/ Ventas es una solución comercial orientada a la gestión de clientes, usuarios, roles, interacciones, oportunidades, propuestas, dashboards y consultas documentales mediante un componente IA RAG parcial. La operación del sistema depende de una interacción coordinada entre frontend React/Vite, backend FastAPI, Firebase Auth, Firebase Admin, Firestore, configuración de entorno, dominios permitidos y proveedor externo de IA cuando corresponde.

La solución de problemas debe ejecutarse con un método de descarte ordenado: primero validar disponibilidad del sistema, luego sesión, luego rol, luego datos, luego reglas de negocio y finalmente componentes externos o parciales. Este orden evita clasificar como error técnico situaciones que corresponden a permisos esperados, falta de datos, configuración incompleta, ambiente pendiente o alcance parcial.

1.1 Objetivos

- Identificar causas probables de errores frecuentes en frontend, backend, autenticación, permisos, datos y componentes parciales.
- Definir acciones correctivas de primer nivel antes de escalar al equipo técnico.
- Establecer evidencia mínima para reproducir y cerrar incidentes.
- Separar bugs reales de restricciones de seguridad, pendientes UAT, exclusiones y cambios de alcance.
- Proteger credenciales, tokens, datos comerciales y configuración sensible durante el diagnóstico.

1.2 Alcance de esta guía

Criterio de alcance

Esta guía cubre el diagnóstico del sistema entregado y documentado. No incorpora soporte productivo indefinido, monitoreo 24/7, corrección de ambientes externos no provistos, evolución funcional, nuevas integraciones, publicación de app móvil ni mejoras del componente IA fuera del estado parcial aceptado.

2. Principios de diagnóstico

Antes de aplicar una corrección se debe distinguir entre error reproducible, restricción esperada, configuración pendiente, dato inexistente o solicitud nueva. El diagnóstico debe conservar evidencia, no exponer secretos y respetar los límites del cierre técnico.

Principio	Aplicación práctica
Reproducibilidad	Un problema debe poder repetirse con pasos concretos, cuenta, rol, pantalla y horario aproximado.
Menor privilegio	Si un usuario no ve datos ajenos, puede ser una restricción correcta y no un defecto.
Backend como autoridad	El frontend puede ocultar acciones, pero la decisión final de acceso reside en el backend.
Ambiente primero	Problemas de DNS, CORS, variables, Firebase o proveedor externo deben validarse antes de modificar código.
Dato antes que bug	Una pantalla vacía puede deberse a datos no cargados, filtros activos, vendedor no asignado o estado inactivo.
Parcialidad declarada	RAG y Flutter deben evaluarse según su estado parcial, no como módulos completos.

Principio	Aplicación práctica
No exponer secretos	No capturar ni compartir API keys, service accounts, tokens completos, contraseñas o archivos de credenciales.

3. Matriz general de problemas frecuentes

La siguiente matriz permite una revisión inicial de síntomas frecuentes. La acción indicada no reemplaza la investigación técnica cuando el caso sea masivo, crítico o no reproducible con facilidad.

Síntoma	Causa probable	Acción inicial
El frontend no carga	Deploy fallido, URL incorrecta, DNS, cache o build roto.	Abrir URL en incógnito, revisar deploy, ejecutar build y validar consola.
Login Google falla	Dominio no autorizado, Firebase mal configurado o popup bloqueado.	Agregar dominio en Firebase Auth, revisar variables VITE_FIREBASE_* y permitir popup.
Error CORS	Dominio del frontend no autorizado por backend.	Agregar origen en CORS_ORIGINS o CORS_ORIGIN_REGEX y redeploy backend.
Usuario recibe 401	Token ausente, expirado o inválido.	Cerrar sesión, volver a ingresar con Google y verificar Authorization Bearer.
Usuario recibe 403	Rol insuficiente, usuario inactivo, acceso webApp/appMovil bloqueado o registro ajeno.	Revisar documento users en Firestore y propiedad del registro.
Usuario no puede entrar	activo=false, rol sin_acceso o acceso de plataforma deshabilitado.	Activar usuario o asignar rol si corresponde según aprobación administrativa.
/readiness no está ready	Variable faltante, placeholder, credencial insegura o ambiente incompleto.	Corregir variables y validar nuevamente readiness.
CSV rechazado	Archivo inválido, encoding incorrecto, columnas no soportadas, duplicados o fórmula injection.	Revisar plantilla, UTF-8, encabezados y errores por fila.
Dashboard sin métricas	Faltan datos, no hay oportunidades/propuestas o vendedor no tiene registros propios.	Confirmar datos base y asignaciones.
Enci Chat no responde	Falta DEEPSEEK_API_KEY, proveedor externo no disponible, rate limit o corpus sin contexto.	Validar backend, variables RAG, documentos indexados y respuesta del proveedor.
App Flutter no cubre un flujo	Limitación de integración parcial o funcionalidad no comprometida.	Clasificar según alcance parcial; registrar como mejora si excede lo acordado.

4. Códigos de error y señales del sistema

Los códigos HTTP y señales internas orientan la primera clasificación del problema. Deben interpretarse junto con el rol del usuario, el endpoint, el ambiente y la acción ejecutada.

Código / señal	Interpretación	Acción recomendada
200 / 201	Operación exitosa.	Validar que la respuesta coincide con lo esperado por negocio.
204	Eliminación o acción sin cuerpo de respuesta.	Confirmar que el registro quedó inactivo o eliminado lógicamente.
400	Solicitud mal formada o archivo no aceptado.	Revisar payload, CSV, encabezados, formato o datos requeridos.
401	No hay autenticación válida.	Renovar sesión, confirmar token y encabezado Authorization.
403	Acceso denegado por rol, estado, plataforma o propiedad.	Revisar rol, activo, webApp, appMovil, vendedorUid y ownerUid.
404	Ruta o recurso no encontrado.	Confirmar URL, ID de recurso, ambiente y exposición de documentación.

Código / señal	Interpretación	Acción recomendada
413	Payload o archivo excede límite.	Reducir archivo, dividir carga o respetar tamaño máximo.
422	Validación de modelo fallida.	Revisar campos requeridos, enums, email, teléfono, valores extra o rangos.
429	Rate limit o cuota excedida.	Esperar ventana de reintento y revisar exceso de solicitudes o cuotas.
503	Servicio de datos o proveedor no disponible.	Escalar con logs, horario y endpoint afectado.
API_NETWORK_FETCH_FAILED	Frontend no pudo contactar la API.	Revisar red, DNS, CORS, VITE_API_BASE_URL y HTTPS.
API_HTTP_	La API respondió con error HTTP.	Revisar código específico y detalle de respuesta.
API_ERROR_BODY_NOT_JSON	Error no retornado como JSON.	Revisar proxy, hosting, caída de servicio o página HTML de error.
API_SUCCESS_BODY_NOT_JSON	Respuesta exitosa no parseable.	Revisar contrato del endpoint y transformación de respuesta.

5. Diagnóstico de autenticación y acceso

El ingreso al sistema depende de Firebase Auth, configuración de dominio autorizado, obtención de ID Token, envío del token al backend y existencia del usuario en Firestore con estado y permisos correctos.

Validación	Qué revisar	Resultado esperado
Dominio autorizado	Firebase Authentication > Settings > Authorized domains.	El dominio del frontend está autorizado.
Variables frontend	VITE_FIREBASE_API_KEY, AUTH_DOMAIN, PROJECT_ID, APP_ID y otras VITE_FIREBASE_*	El build contiene configuración pública correcta.
Token Firebase	El usuario obtiene ID Token tras login con Google.	El frontend envía Authorization: Bearer .
Usuario Firestore	Documento users/{uid}.	Existe usuario con activo=true y rol operativo.
Acceso de plataforma	Campos webApp y appMovil.	El usuario tiene habilitada la plataforma usada.
Rol canónico	sin_acceso, vendedor, supervisor o admin.	Rol distinto de sin_acceso para operar.

5.1 Casos típicos de acceso

Caso	Diagnóstico	Resolución
Popup de Google no abre	Bloqueo del navegador, extensión o política del dispositivo.	Permitir popups o probar en navegador limpio.
Login funciona pero vuelve a login	Token no se guarda en sesión o llamada /auth/login falla.	Revisar consola, Network y respuesta de API.
Usuario creado sin acceso	Alta automática con rol sin_acceso por seguridad.	Administrador asigna rol y habilita webApp/appMovil.
Usuario activo pero bloqueado	activo=false o plataforma deshabilitada.	Corregir campo administrativo si corresponde.

6. Diagnóstico de permisos y roles

Las restricciones por rol y propiedad no deben considerarse error si el bloqueo coincide con la política de seguridad. El vendedor opera registros propios; supervisor y admin tienen mayor visibilidad según

endpoint.

Problema	Validación principal	Criterio
Vendedor no ve cliente	Comparar uid del usuario con vendedorUid u ownerUid del cliente.	Si no coincide, el bloqueo es esperado.
Vendedor no edita cliente	Confirmar propiedad y rol vendedor.	Puede editar solo dentro de su alcance.
Supervisor no cambia usuario	Revisar endpoint usado y rol requerido.	Cambios sensibles pertenecen a admin.
Admin no importa CSV	Confirmar rol admin y endpoint /clientes/import-csv.	Solo admin puede importar.
Usuario inactivo accede parcialmente	Revisar activo=false y caché de sesión.	Debe bloquearse en backend aunque exista token.
Acceso móvil falla	Revisar appMovil en users/{uid}.	Debe estar habilitado para app móvil.

Regla de seguridad

Si la API responde 403 ante un usuario que intenta operar registros ajenos, el sistema está protegiendo información comercial. La corrección no debe consistir en relajar permisos, sino en revisar asignación de datos o rol autorizado.

7. Diagnóstico del frontend web

Los problemas del frontend deben revisarse desde la URL, build, variables VITE_*, consola del navegador, pestaña Network y compatibilidad del navegador. Cuando el frontend está desplegado como aplicación estática, cualquier cambio de variable requiere rebuild o redeploy.

Síntoma frontend	Revisión	Acción
Pantalla blanca	Consola del navegador, errores JS, archivo estático faltante.	Ejecutar npm run build y revisar deploy.
No conecta con API	VITE_API_BASE_URL, HTTPS, DNS, CORS, certificado.	Corregir URL pública de backend y redeploy.
Ruta devuelve 404 en refresh	Configuración SPA en hosting.	Agregar rewrites a index.html.
Login no persiste	Estado de AuthProvider, sessionStorage y token actual.	Cerrar sesión, limpiar caché y probar nuevamente.
Botón no aparece por rol	Regla visual por rol y respuesta /me.	Confirmar que el rol del usuario sea el esperado.
Vista carga sin datos	Network tab, filtros activos, respuesta vacía o 403.	Distinguir dato inexistente de error de API.

7.1 Revisión mínima en navegador

- Abrir DevTools y revisar Console.
- Abrir Network y repetir la acción afectada.
- Confirmar endpoint llamado, status HTTP y cuerpo de respuesta.
- Confirmar que Authorization contiene Bearer y no está vacío.
- Confirmar que la URL base corresponde al backend correcto.
- Probar en ventana incógnita para descartar caché o extensiones.

8. Diagnóstico del backend API

El backend concentra autenticación validada, autorización, reglas de negocio, lectura/escritura en Firestore, documentación Swagger controlada, readiness y endpoints RAG. Si el frontend falla, el backend debe probarse directamente antes de asumir defecto visual.

Área	Validación	Criterio esperado
Disponibilidad	GET /health.	Respuesta status ok.
Configuración	GET /readiness.	Estado ready o checks detallados para corregir.
Autenticación	GET /me con token válido.	Retorna uid, email, rol y activo.
Clientes	GET /clientes con rol vendedor.	Lista filtrada por vendedor si aplica.
Usuarios	GET /users/ con supervisor/admin.	Retorna usuarios según permiso.
Dashboard	GET /dashboard/vendedor o /dashboard/supervisor.	Métricas según rol y datos existentes.
RAG	POST /rag/chat.	Respuesta con fuentes si hay contexto o respuesta local sin contexto.
Swagger	/docs y /openapi.json.	Disponibles solo en ambientes no productivos según configuración.

8.1 Validaciones de ambiente backend

- APP_ENV debe representar el ambiente correcto: development, uat, staging o production según uso.
- FIREBASE_PROJECT_ID debe apuntar al proyecto Firebase real del ambiente.
- GOOGLE_APPLICATION_CREDENTIALS o FIREBASE_SERVICE_ACCOUNT_JSON debe existir solo en backend.
- CORS_ORIGINS debe contener dominios de frontend autorizados; en producción no usar wildcard abierto.
- DEEPSEEK_API_KEY debe existir solo si se requiere operar IA RAG parcial.
- GCS_BUCKET_DOCUMENTS o almacenamiento local deben estar definidos según ambiente de RAG.

9. Diagnóstico de datos e importación CSV

Los errores de datos suelen originarse en asignaciones, duplicados, campos obligatorios, estados inválidos, filtros activos o carga CSV incompleta. La importación masiva debe tratarse como operación administrativa controlada.

Problema	Causa probable	Corrección
Archivo supera límite	CSV mayor a 1 MB o más de 1.000 filas.	Reducir archivo o dividir carga.
Columnas no soportadas	Encabezados distintos a los aceptados.	Usar solo nombre, empresa, email, telefono, rubro, region, estado, vendedorUid, ownerUid.
Encoding inválido	Archivo no está en UTF-8.	Guardar como UTF-8 o UTF-8 con BOM.
Email duplicado	Correo repetido en archivo o ya existente en CRM.	Eliminar duplicado o actualizar manualmente si corresponde.
Fórmula injection	Campo inicia con =, +, - o @.	Eliminar carácter inicial peligroso.
Teléfono rechazado	Número no cumple formato esperado.	Usar número móvil válido, sin caracteres ambiguos.
Cliente no visible tras importar	vendedorUid u ownerUid no asignado al vendedor correcto.	Corregir asignación o revisar rol supervisor/admin.

Problema	Causa probable	Corrección
Estado no reconocido	Valor distinto de En proceso, Completado o Inactivo.	Usar estados canónicos.

9.1 Plantilla de encabezados CSV

nombre, empresa, email, telefono, rubro, region, estado, vendedorUid, ownerUid

10. Diagnóstico de IA RAG parcial

El componente Enci Chat / IA RAG se entrega en estado parcial. Su diagnóstico debe confirmar token, rol, endpoint, proveedor, documentos indexados, límites de uso y disponibilidad de contexto. La ausencia de respuesta especializada puede ser una limitación esperada cuando no existe evidencia suficiente en el corpus.

Síntoma RAG	Revisión	Criterio
Chat responde sin fuentes	Pregunta fuera de corpus o sin documentos indexados relevantes.	No necesariamente es bug; puede ser falta de contexto.
Chat no responde	DEEPSEEK_API_KEY, proveedor externo, logs backend y endpoint /rag/chat.	Escalar si el proveedor o backend falla.
Error 403	Rol mínimo requerido y acceso de usuario.	Debe tener rol operativo permitido.
Error 422	Pregunta vacía o mayor al límite permitido.	Reducir o corregir pregunta.
Error 429	Rate limit por usuario o exceso de solicitudes.	Esperar ventana y reintentar.
Documento no se carga	Rol admin, extensión, tamaño, firma/contenido y almacenamiento.	Corregir archivo o permisos.
Respuestas de baja calidad	Limitación de prompts, corpus, proveedor o tuning.	Registrar como mejora posterior, no como incumplimiento base.

Límite de entrega RAG

No se considera parte de esta guía exigir reentrenamiento, cambio de proveedor, mejora de prompts, evaluación avanzada de calidad, SLA del proveedor, carga masiva de corpus ni optimización semántica no comprometida.

11. Diagnóstico de app Flutter parcial

La aplicación Flutter nativa Android se trata como componente parcialmente realizado y desarrollado fuera del monorepo principal. Su revisión debe basarse en el estado conversado y validado con Max, sin exigir trazabilidad GitHub del código Flutter ni cobertura total de backend si no fue comprometida.

Síntoma Flutter	Revisión	Clasificación
APK no instala	Versión Android, permisos de instalación, firma o compatibilidad del dispositivo.	Problema de distribución local si no fue publicada en tienda.
Login Firebase falla	Configuración Firebase, SHA, google-services, dominio/proyecto y cuenta usada.	Incidente de configuración si afecta función demostrada.
Pantalla existe pero no consume backend	Verificar si ese consumo estaba dentro de la integración parcial.	Limitación parcial o mejora posterior.
Nueva pantalla solicitada	Comparar con funciones demostradas y alcance de Hito 5.	Cambio de alcance.
Publicación en tienda solicitada	Google Play, App Store, firma, certificados o MDM.	Fuera del alcance parcial.

Síntoma Flutter	Revisión	Clasificación
Soporte post-entrega	Mantenimiento, correcciones futuras o evolución.	Requiere nuevo acuerdo.

12. Validaciones rápidas y comandos útiles

Estas validaciones permiten aislar si el problema está en disponibilidad, configuración, build, pruebas, frontend, backend o ambiente. Deben ejecutarse con cuidado para no exponer secretos en capturas o registros compartidos.

12.1 Backend

```
cd Backend
uv sync
uv run uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload
uv run pytest
```

12.2 Frontend web

```
cd frontend
npm ci
npm run lint
npm run build
npm run dev -- --host 127.0.0.1 --port 5173
```

12.3 Validación HTTP

```
Invoke-WebRequest http://127.0.0.1:8000/health
Invoke-WebRequest http://127.0.0.1:8000/readiness
curl https://api.example.com/health
curl https://api.example.com/readiness
```

12.4 Revisión en Swagger

- Abrir /docs en ambiente no productivo.
- Iniciar sesión con Google desde la consola de pruebas si está habilitada.
- Verificar que el esquema HTTPBearer reciba token.
- Probar /me antes de probar endpoints de negocio.
- Probar /clientes, /dashboard, /users o /rag según el rol del usuario.

13. Criterios de escalamiento

No todos los problemas requieren intervención técnica inmediata. Debe escalarse cuando el impacto supere la orientación funcional, cuando exista riesgo de datos, cuando impida flujos críticos o cuando el diagnóstico inicial no permita aislar la causa.

Escalar cuando	Evidencia mínima requerida
Afecta a más de un usuario.	Lista de usuarios, roles, horario y pantallas afectadas.
Impide login o uso completo del sistema.	Captura, consola, Network, estado /health y /readiness.
Involucra pérdida, corrupción o exposición de datos.	IDs de registros, usuario, acción, hora y logs disponibles.
Ocurre después de un despliegue.	Versión, commit o fecha de deploy, ambiente y cambio aplicado.

Escalar cuando	Evidencia mínima requerida
API responde 5xx.	Endpoint, payload anonimizado, hora, ambiente y logs.
RAG falla para todos los usuarios.	Estado de proveedor, variable backend, endpoint /rag/chat y logs.
Flutter falla en función demostrada.	Dispositivo, versión Android, APK usado, pantalla y pasos exactos.

13.1 Severidad de referencia

Severidad	Definición	Acción
S1 Bloqueante	Impide login, acceso general, operación crítica o compromete datos.	Escalamiento inmediato; no liberar salida productiva.
S2 Alta	Impide flujo comercial crítico con workaround limitado.	Escalar y resolver antes de cierre salvo aceptación formal.
S3 Media	Afecta productividad pero tiene workaround claro.	Planificar corrección o documentar workaround.
S4 Baja	Texto, UX menor, ajuste documental o consulta de uso.	Resolver en ciclo normal o registrar mejora.

14. Cierre y registro del problema

Todo problema debe cerrarse con una decisión explícita. El cierre puede ser solución aplicada, workaround entregado, comportamiento esperado, limitación parcial, pendiente del cliente, cambio de alcance o escalamiento abierto.

Estado final	Cuándo usarlo
Resuelto	Se aplicó corrección y el usuario puede continuar.
Workaround entregado	Existe forma alternativa de operar sin bloquear el flujo.
Comportamiento esperado	La respuesta del sistema coincide con roles, seguridad o reglas definidas.
Pendiente cliente/UAT	Depende de ambiente, credenciales, dominios, usuarios reales o validación del cliente.
Limitación parcial	Corresponde a IA RAG parcial o Flutter parcial según alcance aceptado.
Cambio de alcance	Requiere nueva estimación, acuerdo o fase posterior.
Escalado	Requiere análisis técnico adicional y queda con responsable asignado.

14.1 Registro mínimo de cierre

ID de caso:
 Usuario afectado:
 Rol:
 Modulo:
 Sintoma:
 Causa identificada:
 Accion aplicada:
 Estado final:
 Responsable:
 Evidencia de cierre:
 Fecha y hora:

Cierre documental

Si el problema corresponde a una exclusión, componente parcial o pendiente UAT, el cierre debe indicarlo expresamente. No debe quedar redactado como bug pendiente si en realidad corresponde a alcance no comprometido.

Control de responsabilidad documental

Este documento forma parte del paquete de entrega técnica preparado por el Grupo PLM UTEM para el proyecto Enci/ Ventas. Su contenido está orientado a facilitar diagnóstico y soporte de cierre, manteniendo separación entre operación del MVP, validación UAT, componentes parcialmente entregados y solicitudes futuras.